

No weight-four checks for $[[11, 3, 3]]$: a certified computation of optimal check weights of small stabilizer codes

University of Michigan, Ann Arbor, MI, USA

The check weight of a stabilizer code, the largest weight of a generator in a generating set of its stabilizer group, governs the cost of syndrome extraction. Wei, Han, He, Li and Liu recently determined the optimal check weight $W_{\text{opt}}(n, k, d)$ of all qubit stabilizer codes with $n \leq 9$ and left several entries with $10 \leq n \leq 12$ open. We prove that no $[[11, 3, d \geq 3]]$ stabilizer code admits a generating set of weight at most four, so that $W_{\text{opt}}(11, 3, 3) = 5$. This answers a question recorded in the QIQCOP open-problem collection. We also give a $[[12, 3, 3]]$ code with weight-four generators, which shows that 12 is the smallest length of an $[[n, 3, 3]]$ code with weight-four checks, and we determine six further entries: $W_{\text{opt}}(10, 4, 3) = 6$, $W_{\text{opt}}(11, 4, 3) = 5$, $W_{\text{opt}}(11, 5, 3) = 7$, $W_{\text{opt}}(12, 4, 3) = 5$, $W_{\text{opt}}(12, 5, 3) = 6$ and $W_{\text{opt}}(12, 6, 3) = 8$. The nonexistence proofs reduce the problem, through a syndrome formulation, an induction on the length and a normal form under local Clifford operations and qubit permutations, to finitely many Boolean satisfiability instances. Every instance is refuted with a DRAT certificate checked by an independent checker. The complete argument for $[[11, 3, 3]]$, including every reduction and every satisfiability instance, is formalized in Lean 4, and all new constructions are verified in Lean by kernel computation.

1 Introduction

Stabilizer codes [1, 2] are specified by commuting Pauli operators, the checks, whose eigenvalues are measured to detect errors. The weight of a check, the number of qubits on which it acts, determines the size of the measurement circuit and how errors spread during syndrome extraction. Low-weight checks are therefore a central requirement in fault-tolerant architectures and in the theory of quantum low-density parity-check codes [3], and constraints such as two-dimensional geometric locality are known to limit what low-weight codes can achieve [4].

Wei, Han, He, Li and Liu [5] initiated a systematic theory of the *optimal check weight*. This is the quantity $W_{\text{opt}}(n, k, d)$, defined as the least w such that some $[[n, k, d]]$ stabilizer code has a generating set of weight at most w . They showed that computing the check weight of a given code is NP-hard, proved analytical lower bounds, and combined a linear program for weight enumerators with explicit constructions to determine $W_{\text{opt}}(n, k, d)$ for all $n \leq 9$. For $10 \leq n \leq 12$ several entries of their table remained open. One of them, whether $W_{\text{opt}}(11, 3, 3)$ equals 4 or 5, is recorded as an open problem in the QIQCOP collection [6]. A weight-five $[[11, 3, 3]]$ code is obtained from a $[[9, 3, 3]]$ code with weight-five generators by adding two qubits with single-qubit checks [5], and neither the linear-programming bounds nor the known constructions decide whether weight four is possible.

In this paper we settle this and several related entries.

Theorem 1. *No qubit stabilizer code with parameters $[[11, 3, d]]$, $d \geq 3$, has a generating set whose generators all have weight at most four. Hence $W_{\text{opt}}(11, 3, 3) = 5$.*

*These authors contributed equally.

Theorem 2. *There are stabilizer codes with parameters $[[12, 3, 3]]$ and weight-four generators, $[[10, 4, 3]]$ with weight six, $[[11, 4, 3]]$ with weight five, $[[11, 5, 3]]$ with weight seven, $[[12, 4, 3]]$ with weight five, $[[12, 5, 3]]$ with weight six and $[[12, 6, 3]]$ with weight eight. Moreover, no $[[11, 5, 3]]$ code has generators of weight at most six.*

Together with the lower bounds of Ref. [5], Theorems 1 and 2 determine all eight distance-three entries with $10 \leq n \leq 12$ that were open in Ref. [5] (Table 1). Since W_{opt} is nonincreasing in n [5], Theorem 1 together with the $[[12, 3, 3]]$ code shows that the least length of an $[[n, 3, 3]]$ code with weight-four checks is exactly 12. Previously it was only known to lie between 11 and 15 [5], and the QIQCOP entry notes that a negative answer for $n = 11$ “would leave the corresponding questions for $12 \leq n \leq 14$ open” [6].

Method and verification. Constructions are easy to check. Nonexistence is the hard direction, since the naive search space consists of all eight-tuples of the 31 714 Pauli operators of weight at most four on eleven qubits, about 10^{36} tuples. Our argument has three ingredients.

1. *A syndrome formulation.* Every single-qubit error has a syndrome in $\mathbb{F}_2^{n-k}$, and the distance condition becomes a statement about coincidences between these syndromes (Section 2).
2. *Reductions.* A code whose stabilizer contains a weight-one element reduces to a shorter code. If it contains weight-two elements, a basis of their span can be taken as the first generators and placed on fixed qubits. Local Clifford operations and qubit permutations bring the remaining data to a normal form (Section 3).
3. *Certified satisfiability solving.* The remaining finitely many cases are encoded as Boolean formulas and refuted by a SAT solver, which emits proof certificates that are checked independently (Section 4).

Computer-assisted proofs of this kind are established in combinatorics, for example for the Boolean Pythagorean triples problem [7] and Keller’s conjecture [8]. The weak points of such proofs are usually the hand-written reductions and encodings, not the solver. We therefore also formalized the complete argument for Theorem 1 in the Lean 4 proof assistant [9, 10]. The formal proof starts from the definition of a stabilizer code in Definition 1, proves every reduction, and discharges each satisfiability instance with Lean’s verified bit-blasting and certificate-checking tactic `bv_decide` (Section 5). As a sanity check, our encoders reproduce all thirteen distance-three values of Ref. [5] that we tested.

2 Stabilizer codes and syndromes

An n -qubit Pauli operator modulo phase is a vector $v = (a, b) \in \mathbb{F}_2^{2n}$, representing $X^a Z^b = \bigotimes_j X^{a_j} Z^{b_j}$. Its weight is $\text{wt}(v) = |\{j : (a_j, b_j) \neq (0, 0)\}|$, and two Pauli operators commute if and only if the symplectic form $\langle (a, b), (a', b') \rangle = a \cdot b' + b \cdot a'$ vanishes. Following Ref. [6] we use the following definition, which includes degenerate codes.

Definition 1 (Codes with bounded check weight). Let $m = n - k$. Vectors $g_1, \dots, g_m \in \mathbb{F}_2^{2n}$ define an $[[n, k, d \geq 3]]$ stabilizer code with check weight at most w if (i) they are linearly independent; (ii) $\text{wt}(g_i) \leq w$ for all i ; (iii) $\langle g_i, g_{i'} \rangle = 0$ for all i, i' ; and (iv) every v with $1 \leq \text{wt}(v) \leq 2$ and $\langle v, g_i \rangle = 0$ for all i lies in the span $S = \text{span}\{g_1, \dots, g_m\}$.

Condition (iv) says that every Pauli operator of weight at most two that commutes with the stabilizer is itself a stabilizer element up to phase, which is the condition $d \geq 3$ [11].

Remark 3 (Relation to the definition of Ref. [5]). Ref. [5] defines $W_{\text{opt}}(n, k, d)$ over codes of distance exactly d and minimizes over arbitrary, possibly redundant, generating sets. The two definitions agree on every entry we report. An independent subset of a generating set has the same span and no larger weight, so independence is no restriction. A nonexistence statement for $d \geq 3$ implies the one for $d = 3$. All our constructions have distance exactly three: besides $d \geq 3$, we verified by exhaustive search that each has a nontrivial logical operator of weight three.

For a qubit j and a Pauli type $t \in \{X, Y, Z\}$, let $e_{j,t}$ be the single-qubit error of type t on qubit j . Its *syndrome* is $s_{j,t} = (\langle e_{j,t}, g_i \rangle)_{i=1}^m \in \mathbb{F}_2^m$. Writing x_{ij} and z_{ij} for the X - and Z -bits of g_i on qubit j , we have $s_{j,Z} = (x_{ij})_i$, $s_{j,X} = (z_{ij})_i$ and $s_{j,Y} = s_{j,X} + s_{j,Z}$. Two errors $e_{j,t}$ and $e_{l,s}$ have equal syndromes exactly when their product commutes with every generator.

Lemma 4 (Syndrome form of the distance condition). *Suppose (i)–(iii) of Definition 1 hold and S contains no element of weight one. Then (iv) holds if and only if (a) every syndrome $s_{j,t}$ is nonzero, which implies that (b) the three syndromes on each qubit are pairwise distinct, and (c) whenever $s_{j,t} = s_{l,s}$ with $j \neq l$, the weight-two operator $e_{j,t} + e_{l,s}$ lies in S . If S contains no element of weight at most two, (iv) is equivalent to all $3n$ syndromes being nonzero and pairwise distinct.*

Proof. Operators of weight one are the $e_{j,t}$, and $e_{j,t}$ commutes with S iff $s_{j,t} = 0$. Since S has no weight-one elements, (iv) for weight one is exactly (a). Because $s_{j,Y} = s_{j,X} + s_{j,Z}$, condition (a) is equivalent to (a) together with (b). Operators of weight two are the $e_{j,t} + e_{l,s}$ with $j \neq l$, and such an operator commutes with S iff $s_{j,t} = s_{l,s}$; so (iv) for weight two is (c). In the pure case (c) forbids all coincidences between different qubits. $\square$

In the pure case the three syndromes on qubit j are the nonzero vectors of a two-dimensional subspace $V_j \subseteq \mathbb{F}_2^m$, and the distance condition says that $V_1, \dots, V_n$ intersect pairwise trivially. This reformulation removes all auxiliary span-membership variables from the pure case and makes the satisfiability instances much smaller.

3 Reductions and normal forms

Lemma 5 (Weight-one reduction). *If generators satisfying Definition 1 for $(n+1, k, w)$ exist and S contains an element of weight one, then generators satisfying Definition 1 for (n, k, w) exist.*

Proof. Let $e \in S$ be supported on qubit j_0 . Every element of S commutes with e , so its component on j_0 is 0 or e_{j_0} . Deleting qubit j_0 from all elements of S gives a subspace S' of dimension $m-1$, because the kernel of the deletion map on S is spanned by e . Choose a basis of S' among the deletions of the g_i ; weights do not increase. Commutation is inherited because the j_0 -components pairwise commute. If v' on n qubits has weight one or two and commutes with S' , its extension by the identity on j_0 commutes with S , hence lies in S , and deleting j_0 shows $v' \in S'$. Finally $k = (n+1) - m = n - (m-1)$. $\square$

Lemma 6 (Basis exchange). *Let $g_1, \dots, g_m$ satisfy Definition 1 and let $b_1, \dots, b_r \in S$ be linearly independent with $\text{wt}(b_i) \leq w$. Then there are generators $g'_1, \dots, g'_m$ satisfying Definition 1 with the same span S , such that $g'_i = b_i$ for $i \leq r$ and every g'_i with $i > r$ is one of the g_j .*

Proof. Extend $\{b_i\}$ to a basis of S using elements of $\{g_j\}$ (Steinitz exchange). Weights are at most w , the span is unchanged, and so are (iii) and (iv). $\square$

A local Clifford operation on qubit j acts on the pairs (x_{ij}, z_{ij}) by an invertible 2×2 matrix over $\mathbb{F}_2$ and preserves weights and the symplectic form. On the syndromes of qubit j it permutes $\{s_{j,X}, s_{j,Y}, s_{j,Z}\}$, and since $GL(2, \mathbb{F}_2) \cong S_3$ every permutation occurs. Qubit permutations permute the qubit index. Both operations preserve Definition 1 and map S to the corresponding transformed span.

Lemma 7 (Normal form). *Order $\mathbb{F}_2^m$ lexicographically. If the three syndromes on each qubit are pairwise distinct, then after local Clifford operations $s_{j,Z} < s_{j,X} < s_{j,Y}$ for every j . For any t_0 , a permutation of the qubits $j \geq t_0$ then makes $s_{j,Z} \leq s_{j+1,Z}$ for all $j \geq t_0$.*

Proof. Over $\mathbb{F}_2$ every invertible 2×2 matrix has determinant one, so $GL(2, \mathbb{F}_2) = Sp(2, \mathbb{F}_2)$ and every local linear map preserves the symplectic form. It acts on the three nonzero syndromes of a qubit by an arbitrary permutation, and we choose the one that sorts them. Local maps preserve supports. The second step permutes only qubits $j \geq t_0$ and moves each qubit together with its three syndromes, so the first normalization and any support constraints on qubits $j < t_0$ are kept. $\square$

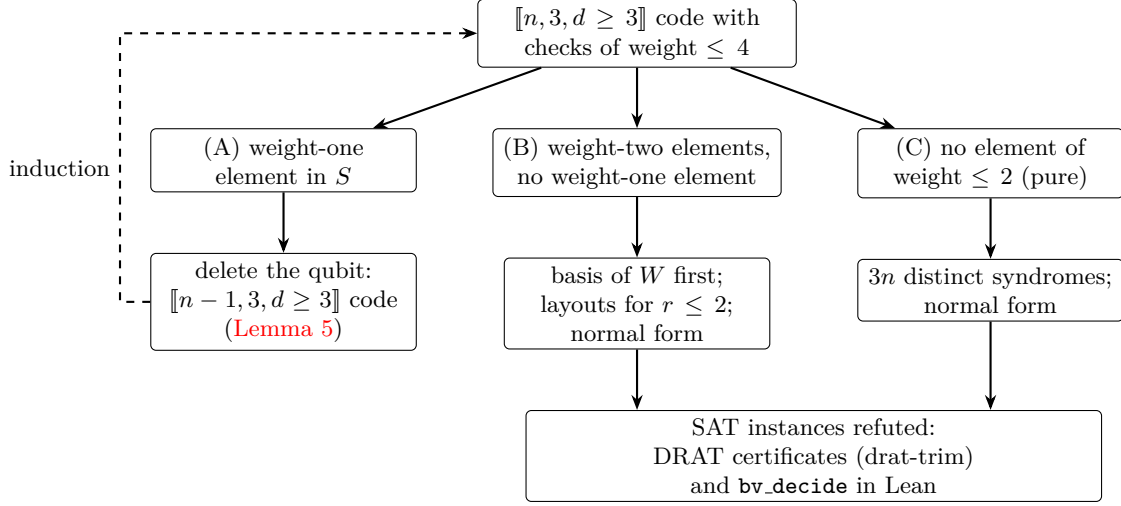


Figure 1: Structure of the proof of [Theorem 1](#). Every code falls into exactly one of three cases. Case (A) reduces the length by one and is handled by induction, down to the trivial base case $n = k$. Cases (B) and (C) are normalized and encoded as Boolean formulas, all of which are unsatisfiable for $n \leq 11$.

We now combine these ingredients. Fix a code as in [Definition 1](#) with $w = 4$ and $n \leq 11$, and consider three exhaustive cases.

- (A) *S contains a weight-one element.* By [Lemma 5](#) a code of length $n - 1$ with the same k exists; we proceed by induction on n . For $n = k$ there are no generators and (iv) fails, which is the base case.
- (B) *S contains weight-two but no weight-one elements.* Let W be the span of the weight-two elements of S and $r = \dim W \geq 1$. Choose a basis of W consisting of weight-two elements and move it to the front with [Lemma 6](#). By [Lemma 4\(c\)](#) every syndrome coincidence between different qubits is then explained by a combination of the first r generators. For $r = 1$ we permute the qubits so that g_1 is supported on $\{1, 2\}$. For $r = 2$ the supports of g_1 and g_2 are equal, share one qubit, or are disjoint, and we place them on $\{1, 2\}$, $\{1, 2\}$, $\{2, 3\}$, or on $\{1, 2\}$, $\{3, 4\}$. In these layouts every element of W is supported on the first $t \leq 4$ qubits, so no coincidence involves a qubit $j > t$, and [Lemma 7](#) applies with strict inequalities beyond t : two qubits $j, l > t$ with $s_{j,Z} = s_{l,Z}$ would give a weight-two element of W supported outside the first t qubits. For $r \geq 3$ we place g_1 on $\{1, 2\}$, require $\text{wt}(g_i) \leq 2$ for $i \leq r$, express collisions through coefficient variables for the first r generators, and sort the qubits $j \geq 3$ non-strictly. The instance with parameter r does not force $\dim W = r$; it only relaxes the constraints, which is sound.
- (C) *S contains no element of weight at most two (pure codes).* By [Lemma 4](#) all $3n$ syndromes are distinct and nonzero, and [Lemma 7](#) gives a strictly sorted normal form.

In each case the normalized code yields an assignment to a Boolean formula that encodes the weight bound, commutation, the syndrome conditions of [Lemma 4](#), the supports fixed above, and the normal form. [Figure 1](#) summarizes the case analysis. In the Lean formalization each length $n \leq 11$ gives one pure instance, four instances for the layouts with $r \leq 2$, and one instance for each $3 \leq r \leq m$. The DRAT pipeline of [Section 4](#) uses a coarser split for $n \leq 9$.

4 Satisfiability instances and certificates

The variables of each instance are the bits x_{ij}, z_{ij} and, in case (B), coefficient bits that express collisions as combinations of the first r generators. Weight bounds are encoded with sequential counters, parities with Tseitin variables, and lexicographic comparisons bitwise. The Python CNF generators are part of the supplementary material.

For [Theorem 1](#) we use, at each length $4 \leq n \leq 11$, one pure instance. For $n \leq 9$ case (B) is encoded as a single instance in which g_1 is a weight-two element on qubits $\{1, 2\}$ and every collision must be explained by a combination of all m generators, which is exactly condition (iv); this only uses [Lemma 6](#) with $r = 1$. For $n = 10, 11$ this unified encoding did not finish within an hour, and we split it into the four layouts with $r \leq 2$ and one instance for each $3 \leq r \leq m$. This gives 33 instances. In the unified instances and the instances with $r \geq 3$ the qubits are not sorted, which only weakens these formulas. All instances were solved with CaDiCaL 3.0.1 [\[12\]](#), which produced DRAT proofs, and all proofs were checked with drat-trim [\[13\]](#) (commit [2e3b2dc](#)). Every instance is unsatisfiable and every certificate verified. On an Apple M1 Max laptop, 16 of the 33 instances take under a second. The hardest is the case $n = 11, r = 3$: solving takes about ten minutes and checking about fourteen minutes. It is followed by the pure case at $n = 11$, which solves in about 1.5 minutes. Exploratory runs used CaDiCaL through PySAT. The nonexistence part of [Theorem 2](#), for $\llbracket 11, 5, 3 \rrbracket$ with weight six, uses the split encoding at lengths $6 \leq n \leq 11$, with base case $n = k = 5$. For $m = 1$ (length six) only the pure instance and the layout with $r = 1$ occur. This gives $2 + 5 + 6 + 7 + 8 + 9 = 37$ instances, all unsatisfiable with verified DRAT certificates.

Cross-validation. With the same pipeline we recomputed thirteen values of $W_{\text{opt}}(n, k, 3)$ that are known from Ref. [\[5\]](#). All of them agree:

(n, k)	(5, 1)	(6, 1)	(7, 1)	(8, 1)	(8, 2)	(8, 3)	(9, 1)	(9, 2)	(9, 3)	(9, 4)	(10, 1)	(10, 2)	(10, 3)
W_{opt}	4	4	4	4	4	6	4	4	5	∞	4	4	5

This tests both directions: the smallest feasible weight must be found, which exercises the satisfiable case, and every smaller weight must be excluded. The value ∞ for (9, 4) means that no code exists even without a weight bound. These runs used the split encoders through PySAT and produced no certificates; they validate the encoders, not the proof.

5 Formal verification in Lean

We formalized [Theorem 1](#) in Lean 4 with Mathlib [\[9, 10\]](#). The final theorem reads

```
theorem no.weight.four.11.3.3 :
  ¬∃ g : Fin 8 → Pauli 11, IsCode 11 8 4 g
```

where `IsCode n m w g` is [Definition 1](#) verbatim: linear independence over $\mathbb{Z}/2$, weights, commutation, and the span condition for all operators of weight one or two. The development has about 1 900 hand-written lines for [Theorem 1](#), 1 600 generated lines for the satisfiability instances and 400 lines for the constructions, and it proceeds exactly as in [Section 3](#). It contains the invariance of [Definition 1](#) under qubit permutations and local linear maps, the basis exchange, the weight-one reduction, the local Clifford normal form and sorting, the translation from a normalized code to Boolean data, and the case analysis (A)–(C) with induction on n .

The Boolean specifications of cases (B) and (C) are Lean definitions with bounded quantifiers. Each of the 61 instances for $4 \leq n \leq 11$ is stated as a Lean theorem and proved by first expanding the quantifiers and then calling `bv_decide`. This tactic bit-blasts the formula inside Lean, calls CaDiCaL, and checks the resulting LRAT certificate [\[14\]](#) with a checker verified in Lean. Lean thus verifies its own Boolean specification and the decomposition into cases; the Python CNF files of [Section 4](#) are an independent second implementation. The Lean instances omit the linear-independence constraint, which only strengthens them. The trusted base consists of the Lean kernel, Mathlib’s definitions, and the Lean compiler and runtime, which evaluate the certificate checker; the last dependency appears as 49 auxiliary axioms in the output of `#print axioms`, one for each instance that is not already closed by the preprocessing (twelve small instances are). The step from W_{opt} to [Definition 1](#) is [Remark 3](#). We used Lean `v4.35.0-rc2` and Mathlib at revision `0653561`. Building the instances for $n = 11$ requires up to about 20 GB of memory and a few hours of CPU time.

All seven constructions of [Theorem 2](#) are verified in Lean by kernel computation (`decide`): commutation, weights, linear independence via an explicit dual basis, and the absence of commuting

g_1						Z	Z			Z	Z	
g_2								Z	Z	X	X	
g_3	Z	Z						X	X			
g_4					Z	Z	X	X				
g_5				Z		X		Y				X
g_6	Z		Z		X	X						
g_7		Z	X	X		Z						
g_8		X		Y					Y		Z	
g_9	X				Y		Z		Z			
	1	2	3	4	5	6	7	8	9	10	11	12
	Qubit											

Figure 2: Generators $g_1, \dots, g_9$ of a $[[12, 3, 3]]$ stabilizer code with checks of weight four. Colored cells show the Pauli operator of each generator on each qubit (blank: identity). The generators commute pairwise and are linearly independent, and the 36 single-qubit syndromes are nonzero and pairwise distinct, so the code is pure with distance at least three; a weight-three logical operator shows that the distance is exactly three.

	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = 6$
$n = 10$	4	4	5	6 (6–7)	∞	∞
$n = 11$	4	4	5 (4–5)	5 (5–7)	7 (6–8)	∞
$n = 12$	4	4	4 (4–5)	5 (5–7)	6 (6–8)	8 (8–10)

Table 1: Optimal check weight $W_{\text{opt}}(n, k, 3)$ for $10 \leq n \leq 12$ and $k \leq 6$. Bold entries were open in Ref. [5], and the previous bounds are given in parentheses. Lower bounds for the bold entries come from Ref. [5], except for $W_{\text{opt}}(11, 3, 3) \geq 5$ (Theorem 1) and $W_{\text{opt}}(11, 5, 3) \geq 7$ (Theorem 2). Upper bounds come from our constructions, except for $W_{\text{opt}}(11, 3, 3) \leq 5$ [5]. Remaining entries are as in Ref. [5].

operators of weight one or two, which gives $d \geq 3$. These theorems depend only on Lean’s standard axioms. The nonexistence of weight-six $[[11, 5, 3]]$ codes is certified by DRAT but not formalized in Lean.

6 Constructions and updated table

The $[[12, 3, 3]]$ code is shown in Fig. 2. It is pure: all 36 single-qubit syndromes are nonzero and pairwise distinct, so its distance is at least 3. It has a nontrivial logical operator of weight three, so the distance is exactly 3 (Remark 3). The other constructions of Theorem 2 are listed in Section A; all of them are pure as well. Table 1 shows the distance-three entries of Table II of Ref. [5] for $10 \leq n \leq 12$ together with the values established here.

7 Discussion

Theorem 1 answers the QIQCOP question [6] negatively, and the $[[12, 3, 3]]$ construction answers the follow-up question for lengths $12 \leq n \leq 14$. The method depends on three features that are common to many finite questions about stabilizer codes: syndromes turn distance conditions into injectivity constraints; low-weight stabilizer elements can be moved to the front and placed on fixed qubits; and the symmetry group of local Clifford operations and qubit permutations admits

a simple normal form. We expect the same approach to apply to other small entries of the check-weight table, including distance four, where the syndrome conditions involve pairs of errors.

The two layers of certification play different roles. DRAT certificates make the solver untrusted but still rely on the correctness of the encoding, which we also cross-validated against thirteen known values. The Lean formalization removes the trust in the encoding and in the reductions and leaves the Lean kernel, Mathlib’s definitions and the Lean compiler. In our view, this combination is a practical way to make computer-assisted results in quantum coding theory checkable.

Acknowledgments

The reduction strategy, the satisfiability encodings and the Lean formalization were developed with the assistance of Claude (Anthropic). The authors checked all arguments and take full responsibility for the content. We thank the maintainers of the QIQCOP collection for curating the problem and F. Wei, Z. Han, A. Y. He, Z. Li and Z.-W. Liu for their table of optimal check weights.

Code availability

The Lean formalization, the SAT encodings, the certificate logs and the scripts are available at <https://github.com/Ivann1242/optimal-check-weight-11-3-3>.

A Generators of the new codes

Each string lists the Pauli operator of one generator on qubits $1, \dots, n$.

$[[10, 4, 3], w = 6$	$[[11, 4, 3], w = 5$	$[[11, 5, 3], w = 7$	$[[12, 4, 3], w = 5$
IZIZIZZZIZ	ZIZZIIIIZZ	IIIZIZZZZZ	IIIIIIIZZZZZ
IIIZZZIZZZ	IZIZIIZZZI	IZZIZIIXXXX	IIIZIIXXXXX
ZIIIIIXXXX	ZIIIZZIIZI	IIIXXXIYXX	IIIZZZXIIIX
ZIZXXIIYY	IIIIIZXXXX	IXXIYXZIXY	IIIXXXIIIIIZ
ZXXZIIIXII	IIIXXXIIYY	ZIXIYZYIZI	IZZZIYXIIIII
XIYIXZYIYY	IXXIIXXIYY	XVIIYYIYIY	IXXIIIIIZZI
	XIYIYIYIYY		ZIZIXIYYIXI
			XIYIIYYIYYIY

$[[12, 3, 3], w = 4$	$[[12, 5, 3], w = 6$	$[[12, 6, 3], w = 8$
IIIIIIIZZIIZ	IZIIZIIZZZZ	IIIIZZZZZZZZ
IIIIIIIZZXX	IIIIIZZXXXX	ZIIZIIXXXXXX
ZZIIIIIXXII	IIZZZXXIIII	IIZXXZIIIXY
IIIZZZXXIIII	IIXXXIIZZY	IZIXYIYYXIY
IIIZIXIYIIIX	IXIYXXYIIII	IXXIIXIYXYX
ZIZIXXIIIIII	ZZIIYYIYIXY	XIYYIYYZYIX
IZXIZIIIIIII	XIZIXIYIZIXI	
IXIYIIIIYYIZ		
XIIYYIZIZIII		

References

- [1] Daniel Gottesman. “Stabilizer codes and quantum error correction”. PhD thesis. California Institute of Technology. (1997). [arXiv:quant-ph/9705052](https://arxiv.org/abs/quant-ph/9705052).
- [2] A. Robert Calderbank, Eric M. Rains, Peter W. Shor, and Neil J. A. Sloane. “Quantum error correction via codes over $GF(4)$ ”. *IEEE Transactions on Information Theory* **44**, 1369–1387 (1998).

- [3] Nikolas P. Breuckmann and Jens Niklas Eberhardt. “Quantum low-density parity-check codes”. *PRX Quantum* **2**, 040101 (2021).
- [4] Sergey Bravyi, David Poulin, and Barbara Terhal. “Tradeoffs for reliable quantum information storage in 2D systems”. *Physical Review Letters* **104**, 050503 (2010).
- [5] Fuchuan Wei, Zhengyi Han, Austin Yubo He, Zimu Li, and Zi-Wen Liu. “Theory of low-weight quantum codes”. arXiv:2601.19848v2 (2026).
- [6] QIQCOP Zoo. “Weight-four generators for an $[[11,3,3]]$ stabilizer code (problem op_458e9e86)”. https://qiqc-op.com/problem/op_458e9e86cbddccb/ (2026). Accessed 24 September 2026.
- [7] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. “Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer”. In *Theory and Applications of Satisfiability Testing – SAT 2016*. Pages 228–245. Lecture Notes in Computer Science (2016).
- [8] Joshua Brakensiek, Marijn J. H. Heule, John Mackey, and David Narváez. “The resolution of Keller’s conjecture”. In *Automated Reasoning – IJCAR 2020*. Pages 48–65. Lecture Notes in Computer Science (2020).
- [9] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 theorem prover and programming language”. In *Automated Deduction – CADE 28*. Pages 625–635. Lecture Notes in Computer Science (2021).
- [10] The mathlib Community. “The Lean mathematical library”. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. Pages 367–381. (2020).
- [11] Emanuel Knill and Raymond Laflamme. “Theory of quantum error-correcting codes”. *Physical Review A* **55**, 900–911 (1997).
- [12] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froykys, and Florian Pollitt. “CaDiCaL 2.0”. In *Computer Aided Verification – CAV 2024*. Pages 133–152. Lecture Notes in Computer Science (2024).
- [13] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. “DRAT-trim: Efficient checking and trimming using expressive clausal proofs”. In *Theory and Applications of Satisfiability Testing – SAT 2014*. Pages 422–429. Lecture Notes in Computer Science (2014).
- [14] Luís Cruz-Filipe, Marijn J. H. Heule, Warren A. Hunt, Matt Kaufmann, and Peter Schneider-Kamp. “Efficient certified RAT verification”. In *Automated Deduction – CADE 26*. Pages 220–236. Lecture Notes in Computer Science (2017).